Lumen

Step-by-step script and recording guide

Target length: 12 to 13 minutes (the assignment asks for 8 to 15). A screen recording with your voice is enough.

Repository: github.com/salman-kv/lumen
Version of this document: 10 November 2026

Contents

What the assignment requires (section 16) and where it is covered 3

Before you press record (10 minutes) 3

Part 1. Introduction (0:00 to 0:45) 3

Part 2. Architecture (0:45 to 3:15) 4

Part 3. Product walkthrough (3:15 to 7:15) 4

Part 4. Questions with references (7:15 to 9:00) 5

Part 5. The AI decision flow (9:00 to 11:30). The most important part 5

Part 6. Two organizations and isolation (11:30 to 13:00) 5

Part 7. Wrap-up (13:00 to 13:45) 6

If something goes wrong while recording 6

How to record (free, Windows) 6

How to submit it 7

Speak slowly and simply. You do not need to read word for word: the "Say" lines are examples in plain English.

What the assignment requires (section 16) and where it is covered

The assignment asks for	Covered in part
Architecture: frontend, backend API, tenant, knowledge base and retrieval, LLM, System 1 models, application logic, final response	Part 2
Registration/login and dashboard	Part 3
Upload and manage PDFs	Part 3
Generate the assistant URL and the embed code	Part 3
Embed the assistant into another web page	Part 3
Questions with document and page references	Part 4
One question that needs several decisions (inputs, decomposition, System 1, parallel steps, combining, final answer)	Part 5
Two organizations with different documents, each using only its own	Part 6

Do not read out private reasoning. Explain the architecture and the data flow instead.

Before you press record (10 minutes)

1. **Start the app** on your PC. Double-click `scripts\update_live.bat` and wait for "Done". Record on `http://localhost:8000` (faster and no tunnel needed). Optionally show the public link at the end.
2. **Check your Groq key** is in `backend\.env`. Open `http://localhost:8000/api/health` and look for `"ok": true`.
3. **Two browser windows**, side by side or in two profiles:
   - Window A: `http://localhost:8000` (you will register a brand-new organization here).
   - Window B: stays on the sign-in page for the demo accounts.
4. **Open in tabs** (so you can switch without searching):
   - `docs\Lumen_Documentation.pdf`, page 4 (architecture diagram) and page 13 (the question pipeline).
   - A terminal in `D:\ishita\app\backend` ready for `uv run pytest tests/test_isolation.py -v`.
   - File Explorer on `D:\ishita\app\sample_data\northfield` (the PDFs to upload).
5. **Clear the guided tour** if you want to show it: in the browser console run `localStorage.removeItem('lumen_tour_done')`, or just skip it.
6. **Pick a new organization name and email** for the walkthrough, for example `Demo University` and `demo-video@example.com` with a password of 8 or more characters.
7. **Close** chat apps and notifications. Set the browser zoom to about 110% so text is easy to read.
8. **Go slowly with questions.** The free model allows about 8,000 tokens a minute. Wait 10 to 15 seconds between questions and do not paste many at once. If an answer looks very short, ask again.

Part 1. Introduction (0:00 to 0:45)

Show: the Lumen landing page.

Say: "Hello, I am [your name]. This is Lumen, a multi-tenant AI Knowledge and Decision Assistant. An organization uploads its policy PDFs and gets an assistant it can share or embed on any website. Visitors ask questions and get answers with the exact document and page. The main idea is that the language model should not do everything. I

split the work into three layers, and I will show how."

Part 2. Architecture (0:45 to 3:15)

Show: the documentation PDF, page 4 (the big picture), then page 13 (the question pipeline).

Say (page 4):

- "The frontend is a React app. It has the admin dashboard and the visitor chat."
- "The backend is one FastAPI server. It exposes the API, serves the website, runs the PDF pipeline, and runs the question engine."
- "Data is in SQLite. Every row has an organization id, and every query filters by it. That is how organizations stay separate."
- "Admins sign in with a signed login token. Visitors use a secret assistant token in the link. That token is not a login, so it can never reach the admin APIs."

Say (three layers):

- "Layer one is the language model, Groq in this setup. It only understands wording and writes the final sentence."
- "Layer two is System 1: small, fast models that make the real decisions. Search embeddings and a reranker, a classifier for the question type, and rule checks."
- "Layer three is normal application code. It routes the message, decides when to refuse, combines the results, and finds missing information."

Say (page 13, the pipeline):

- "A question first goes through a router for greetings and unclear messages. A real question is searched, but only inside that organization's documents."
- "Search is keyword plus meaning, merged, then a reranker scores how well each passage answers the question."
- "If there is no real evidence, the assistant refuses and tells you what it can help with. It never guesses."
- "If the visitor states facts about themselves, I extract those facts, run one yes/no check per rule in parallel, and combine them."

Part 3. Product walkthrough (3:15 to 7:15)

1. **Register (0:40).** In window A click **Create an account**. Type the organization name, email and password. Point out the live validation and the password strength bar. Land on the dashboard. **Say:** "Each registration creates a new, private organization."
2. **Guided tour (0:20, optional).** If it opens, click through two steps and **Skip tour**. **Say:** "New users get a short tour they can skip, and replay from 'Take the tour'."
3. **Upload (1:00).** Go to **Knowledge**. Drag `Scholarship_Rules.pdf` from the Northfield folder. Watch the stages: extracting, indexing, reading rules. Then upload `Fee_Policy.pdf` too. **Say:** "Each PDF is split into passages that never cross a page, embedded, and scanned for rules like 'GPA of at least 3.5'."
4. **Limits and errors (0:30).** Try to upload a `.txt` file and show the clear error message. Point at the "documents used" ring. **Say:** "The limits from the assignment are 10 PDFs, 20 pages, 10 megabytes, and the errors say what went wrong."
5. **What the assistant learned (0:30).** Open the section with the rule cards. **Say:** "These are the rules found in the PDF, each with its page. They are what the yes/no checks run on."
6. **Assistant tab (1:00).** Change the name and colour; watch the **live preview** update. Show the **public link** and the **embed code**; click **Regenerate** and cancel the confirmation. **Say:** "The link works without an account.

Regenerating it would stop the old link immediately."

7. **Embed in another page (0:40).** Open `http://localhost:8000/demo.html?token=<your assistant token>` (copy the token from the link after `/a/`). Click the chat bubble. Mention `&mode=inline` for the iframe style. **Say:** "This page stands in for a customer's website. They paste one script line and the assistant appears."

Part 4. Questions with references (7:15 to 9:00)

Ask these in the embedded chat, or in the Assistant tab's live preview.

1. Type **"hi"**. **Say:** "Greetings are answered instantly by the app, with suggestion buttons, no search needed."
2. **"What is the refund policy?"**: click the **source chip** to show the page snippet. **Say:** "Every answer cites the document and page, and I can open the exact passage."
3. **"can i get money back if i quit"**. **Say:** "People do not use policy words. The app maps everyday wording to the policy's words."
4. Click a **follow-up button** such as "Tell me about Fee Waiver". **Say:** "These buttons come from the same document."
5. **"Does the university offer a PhD in astrophysics?"** (use the Northfield demo account if your own organization has no admissions PDF). **Say:** "When the documents do not contain the answer, it says so, names what it can help with, and shows no fake sources."

Part 5. The AI decision flow (9:00 to 11:30). The most important part

*Use the Northfield demo organization (it has all the scholarship PDFs). Click **Northfield University** on the sign-in page, or use its public chat page.*

Ask: **"I'm a second-year student with a 3.6 GPA and I already have a sports scholarship. Am I eligible for the merit scholarship?"**

Open **How I decided** and walk the steps:

1. **Search** (System 1): "Keyword and meaning search, reranked, only this organization's passages."
2. **Classify** (System 1): "A small classifier says this is an eligibility question."
3. **Choose policy** (System 1): "It picks the Merit Scholarship."
4. **Extract facts** (language model plus patterns): "Year 2, GPA 3.6, holds a sports scholarship. A fact the model invents is thrown away if it is not in the question."
5. **Decide** (System 1, in parallel): "One yes/no check for each rule at the same time, plus an evidence score. Two pass, one fails."
6. **Combine** (application logic): "All conditions must pass, so the result is Not eligible."
7. **Answer** (language model): "It only words the result. The reasons are written by the app, and a guard rejects wording that contradicts the verdict."

Then show two more cases quickly:

- **"Am I eligible for the merit scholarship?"** with no facts: "It asks for the missing facts instead of guessing."
- **"I'm a third year student with a 3.8 GPA and no sports scholarship, can I get the merit scholarship?"**: "Now every rule passes."

Part 6. Two organizations and isolation (11:30 to 13:00)

1. Sign out, then click **Acme Corp**. Show different documents, a different assistant colour and a different link.

2. In Acme's chat ask the scholarship question: **"Am I eligible for the Merit Scholarship?"** It cannot find it. **Say:** "Acme has no scholarship documents, and it cannot see Northfield's."
3. Ask Acme **"Can I work remotely?"** and show the answer with its own document.
4. Open the **Usage** tab: the counts, the 7-day chart, and the **Question history**. Click a row to show the answer and sources; mention search, filters and CSV.
5. In the terminal run `uv run pytest tests/test_isolation.py -v` and read two test names aloud: "organization B cannot get organization A's knowledge" and "the assistant token cannot reach the admin API".
6. Optionally run `uv run pytest -q` to show all **119 tests** passing.

Part 7. Wrap-up (13:00 to 13:45)

Show: the GitHub page https://github.com/salman-kv/lumen and the live link.

Say: "To summarise: Lumen separates language from decisions. Small models and plain code decide, and the language model only words the answer. Each organization's data is isolated and tested. The limits are that rule mining is pattern based, rules combine with AND only, and it uses SQLite, so in production I would add Postgres, a job queue and a review screen for rules. Thank you for watching. I enjoyed building this and I would love to keep building products like it with your team."

If something goes wrong while recording

Problem	What to do
An answer is very short or odd	The free model was busy. Wait 15 seconds and ask again. Cut the retake in editing, or keep it and say "let me ask again"
"You're asking too quickly"	Wait one minute, then continue
Server not responding	Run `scripts\update_live.bat` and wait for "Done", then reload the page
You forget a line	Pause, say it again, and trim the gap afterwards. Perfect is not needed
Sign-in fails for a demo account	Make sure the server was started with `SEED_DEMO=true` (the script does this) and wait 30 seconds after start

How to record (free, Windows)

Easiest: Xbox Game Bar (built in). Open the page you will record, press **Win + G**, then **Win + Alt + R** to start and stop. Turn the microphone on in the Game Bar's audio panel. The video is saved in `Videos\Captures`. It records one window at a time, so keep the browser as the active window.

More control: OBS Studio (free, https://obsproject.com). Add a *Display Capture* (or *Window Capture*) and your microphone, then **Start Recording**.

Tips

- Do a 30-second test first and listen back. Check your voice is clear and the text on screen is readable.
- Record in one go or in parts per section. Short parts are easier to redo.
- The free editor in Windows (**Clipchamp**, or **Photos > Video editor**) can trim mistakes and join the parts.
- Export at **1080p**. Keep the file under about 1 GB.

How to submit it

1. **Upload to YouTube** (Studio > Create > Upload) and set visibility to **Unlisted**, or upload to **Google Drive** and set sharing to **"Anyone with the link can view"**.
2. Open the link in a private window to make sure it plays without signing in.
3. Paste the link in your email and in the submission form, and add it to the top of `README.md`.